

Unsantitized HTML / Potential Cross-Site Scripting

Assessment Identifier: **WS-STA-06** | Assessment Type: **Static** | Date: **September 08, 2026**

Severity: HIGH	CVSS: 7.3	CWE: CWE-79
OWASP: A03	WASC ID: N/A	Confidence: High candidate
Target / Component: React/HTML-generating components	Affected URL: https://github.com/worldmonitor/worldmonitor/blob/main/src/components/	Parameter: Dynamic HTML markup rendering sink

1. Description

Static analysis identified HTML-generation patterns where dynamic values may reach HTML rendering contexts. The issue is exploitable only if attacker-controlled data can reach an executable HTML/JavaScript sink without appropriate encoding or sanitization.

2. Steps to Reproduce

1. Identify the current static-analysis finding.
2. Determine whether the value originates from an untrusted source.
3. Trace it to the HTML sink.
4. Use a harmless local XSS test marker.
5. Determine whether it is rendered as HTML or text.

3. Controlled Proof of Concept

Safe Proof of Concept:
 Use only an isolated local test value such as `<img src=x onerror=alert('XSS-TEST')>`. Do not use credential theft, cookie extraction, data exfiltration, or persistent payloads.

4. Business & Security Impact

A confirmed XSS issue could allow attacker-controlled JavaScript to execute in a victim's browser context.

5. Remediation Recommendations

Primary Fix: Prefer React's escaped rendering, avoid dangerouslySetInnerHTML unless necessary, sanitize required HTML, apply context-specific output encoding, and use CSP as defense in depth.

1. Eliminate dangerouslySetInnerHTML usages where standard React JSX string interpolation can be used.
2. When rich HTML rendering is essential, sanitize markup using DOMPurify with strict attribute allowlists.
3. Enforce a restrictive Content Security Policy that bans inline scripts.

6. Evidence & Validation

Evidence Type: Textual / Static Source Analysis
 This finding was identified through static analysis of repository source code and configuration files. Per assessment requirements, no simulated runtime screenshots or dynamic PoC captures are applicable. Verification must be performed via source code review and static analysis tool verification.

Authorization & Safety Compliance: *This security assessment was conducted strictly under authorized parameters in accordance with ethical hacking standards. All validations and safe proof-of-concept tests were non-destructive. No persistent modifications or sensitive data exfiltration occurred.*